الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يونيورسيتي إسلام انتارابغسا ملايسيا

(Company No.: 198301005860)

Garden of Knowledge and Virtue

MECHATRONICS SYSTEM INTEGRATION : MCTA 3203

LABORATORY REPORT

SECTION 1

TITLE : SERIAL INTERFACING WITH MICROCONTROLLER: SENSORS AND ACTUATORS (WEEK 4)

LECTURER'S NAME: ASSOC. PROF. IR. TS. DR. ZULKIFLI BIN ZAINAL ABIDIN

DATE OF EXPERIMENT : 29 OCTOBER 2025

DATE OF SUBMISSION : 5 NOVEMBER 2025

GROUP : 4

NO.	NAME	MATRIC NO.
1	MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	2311779
2	AHMAD HARITH IMRAN BIN MOHD YUSOF	2311581
3	ARIFA AQILAH BINTI ABDUL HALIM	2311530

ACKNOWLEDGEMENTS

We would like to express our heartfelt appreciation to Dr. Zulkifli Bin Zainal Abidin for his continuous guidance, support, and insightful explanations throughout this experiment. His expertise greatly helped us understand the concepts of interfacing and digital control using Arduino. We would also like to thank our lab assistant for providing technical assistance and ensuring that the experiment ran smoothly. Finally, we extend our gratitude to our classmates for their cooperation and teamwork during the lab session.

ABSTRACT

This experiment focused on designing a motion-activated RFID access system using Arduino, an MPU6050 motion sensor, and an RFID module. The main goal was to detect hand motion in real time and integrate it with RFID authentication to control a servo motor and LED indicators. In the first part, the MPU6050 sensor was used to capture accelerometer data and display the hand motion path using Python. The system could recognize circular motion gestures. In the second part, the RFID module was combined with the motion detection feature to allow access only when both the RFID tag was valid and the correct motion pattern was performed. The servo motor unlocked for valid inputs and remained locked for invalid ones. Overall, the experiment demonstrated successful communication between Arduino and Python through serial connection and showed how sensors and actuators can be integrated into a smart control system.

TABLE OF CONTENT

1.0 INTRODUCTION

This experiment was carried out to learn how to connect and communicate different sensors and actuators with a microcontroller using serial communication. The main objective was to build a motion-activated RFID access system that can identify authorized users and respond based on motion detection. The experiment involved two key components which is the MPU6050 sensor that detects hand motion and the RFID module which reads tag information.

In this system, the MPU6050 provides accelerometer and gyroscope data to detect circular motion patterns, while the RFID module verifies the user's identity. Both modules send data to the Arduino, which processes it and controls a servo motor and LEDs as indicators. This project combines hardware and software through Python and Arduino communication.

The expected outcome was that the system would only unlock the servo motor when a valid RFID tag was scanned *and* the correct motion pattern was performed. This experiment helped demonstrate how sensor data and control signals can be integrated to create an intelligent and secure access system.

2.0 MATERIAL AND EQUIPMENT

EXPERIMENT 1

1. Arduino Board
2. MPU6050 Sensor
3. Jumper Wires
4. USB Cable
5. Computer with Arduino IDE and Python installed

EXPERIMENT 2

1. Arduino Board
2. RFID Card Reader
3. RFID Card
4. Servo Motor
5. Red and Green LEDs
6. Resistor
7. MPU6050 sensor
8. Jumper Wires
9. Breadboard

3.0 EXPERIMENTAL SETUP

EXPERIMENT 1

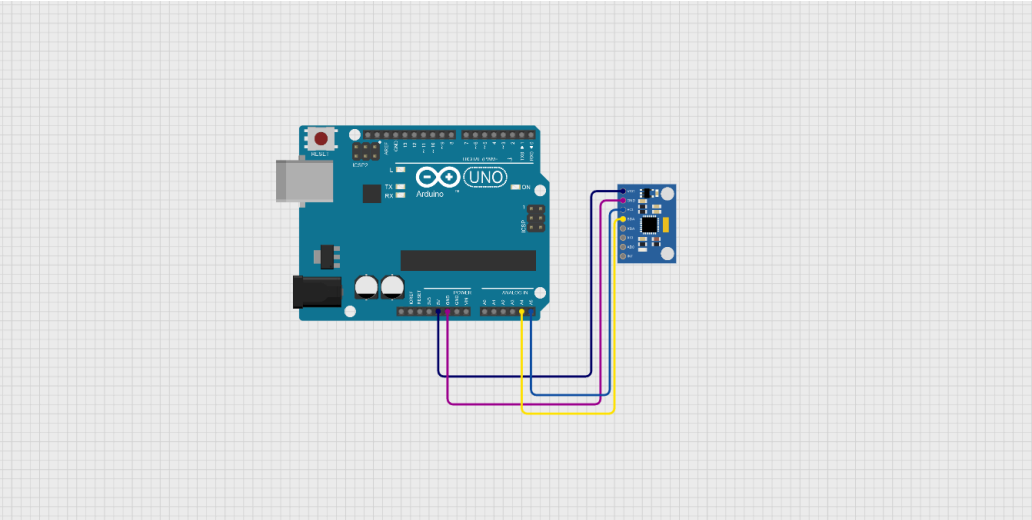


Figure A: Components connected to arduino uno in cirkit designer

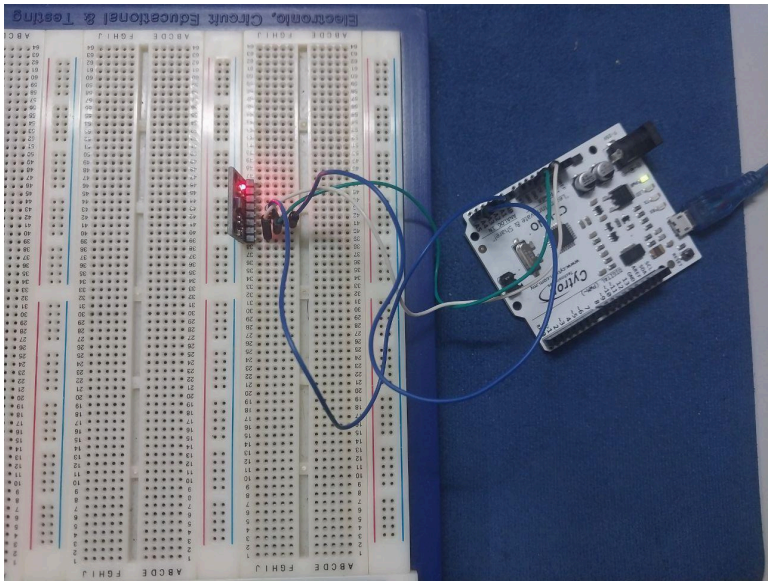


Figure B: Components connected to arduino uno physically

EXPERIMENT 2

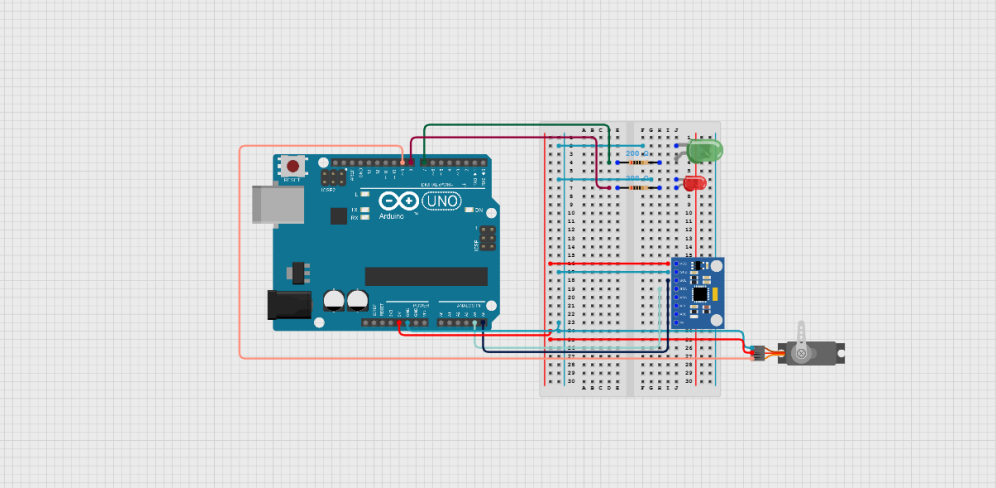


Figure C: Components connected to arduino uno in cirkit designer

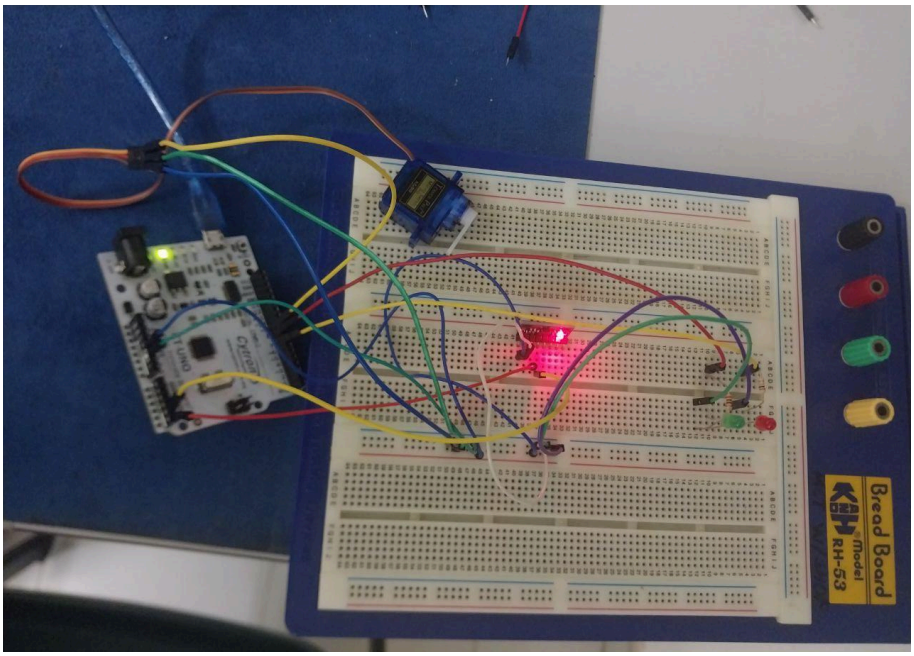


Figure D: Components connected to arduino uno physically

4.0 METHODOLOGY

EXPERIMENT 1

1. The circuit was assembled following the instructions from the lab module.
2. The MPU6050 sensor was connected to the Arduino Uno
3. The Arduino code was uploaded using the Arduino IDE to read accelerometer and gyroscope data.
4. Python was used to receive serial data from Arduino and plot the motion path in real time using matplotlib
5. Circular hand motion was performed to observe changes in the plotted graph.
6. The process was repeated several times to check the accuracy and consistency of the motion detection.

Circuit Assembly

MPU6050 Connections:

- VCC pin is connected to 5V
- GND pin is connected to GND pin of Arduino
- SDA pin is connected to A4 pin of Arduino
- SCL pin is connected to A5 pin of Arduino

Programming Logic

- The Arduino continuously reads acceleration and gyroscope values from the MPU6050.
- The values are sent to the computer via serial communication at a baud rate of 9600.

- The Python program receives and splits the data into X, Y, and Z components.
- The motion path is plotted dynamically in real time using the matplotlib library.
- When circular movement is detected, the shape on the graph confirms correct pattern recognition.

Code Used

- Arduino

```
#include <Wire.h>
#include <MPU6050.h>
MPU6050 mpu;
const int ACCEL_THRESHOLD = 16400;
const int GYRO_THRESHOLD = 1700;
int ax,ay,az,gx,gy,gz;
void setup() {
  Serial.begin(115200);
  Wire.begin();
  mpu.initialize();
}
void loop()
{
  mpu.getAcceleration(&ax, &ay, &az);
  int gesture = detectGesture();
```

```
  Serial.print(ax);
  Serial.print(",");
  Serial.print(ay);
  Serial.print(",");
  Serial.print(az);
  Serial.print(",");
  Serial.println(gesture);
  delay(50);
}
int detectGesture() {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  long accel_magnitude_sq = (long)ax*ax + (long)ay*ay + (long)az*az;
  long gyro_magnitude_sq = (long)gx*gx + (long)gy*gy + (long)gz*gz;
  if (accel_magnitude_sq > (long)ACCEL_THRESHOLD * ACCEL_THRESHOLD &&
gyro_magnitude_sq > (long)GYRO_THRESHOLD * GYRO_THRESHOLD)
  {
    return 1;
  }
  else
  {
    return 0;
  }
```

```
}
```

- Python

```
import serial

import numpy as np

import matplotlib.pyplot as plt

from drawnow import drawnow


arduino = serial.Serial('COM3',115200)

plt.ion()

fig, ax = plt.subplots()

# Data buffers

x_pos = [0]

y_pos = [0]

z_pos = [0]

vx, vy, vz = 0, 0, 0  # Initial velocities

dt = 0.02  # Time step (~20ms)

last_gesture = "None"


def make_plot():

    plt.title("Real-Time 3D Path")

    plt.xlabel("X Position")

    plt.ylabel("Y Position")

    plt.plot(x_pos, y_pos)
```

```
    plt.xlim(-100,100)
    plt.ylim(-100,100)
    plt.grid(True)
while True:
    while arduino.in_waiting == 0:
        pass
    line = arduino.readline().decode('utf-8').strip()
    values = line.split(',')
    if len(values) == 4:
        try:
            ax = int(values[0]) / 16384.0  # Convert to g
            ay = int(values[1]) / 16384.0
            az = int(values[2]) / 16384.0 - 1.0  # Remove gravity
            gesture_status = int(values[3])
            if gesture_status == 1:
                last_gesture = "Circle Gesture"
            else:
                last_gesture = "None"
            print(f"Detected Gesture: {last_gesture}")
            vx += ax * 9.81 * dt
            vy += ay * 9.81 * dt
            vz += az * 9.81 * dt
```

```
        x_pos.append(x_pos[-1] + vx * dt)
        y_pos.append(y_pos[-1] + vy * dt)
        z_pos.append(z_pos[-1] + vz * dt)
        # Limit buffer size
        if len(x_pos) > 200:
            x_pos.pop(0)
            y_pos.pop(0)
            z_pos.pop(0)
        drawnow(make_plot)
    except:
        continue
```

Control Algorithm

1. Initialization
    - Start serial communication at 9600 baud rate.
    - Initialize the MPU6050 sensor for data reading.
2. Continuous Data Collection
    - Read accelerometer values from MPU6050.
    - Send X, Y, and Z values to the computer through serial.
    - Python receives the data and updates the motion plot in real time.
3. Pattern Detection

- Identify circular movement from the plotted path.
- Repeat process for multiple trials to confirm consistency.

EXPERIMENT 2

1. The circuit was assembled following the instructions from the lab module.
2. The MPU6050, servo motor and LEDs were connected to the Arduino using the breadboard.
3. The Arduino code was uploaded to control the servo and LEDs based on commands received from Python.
4. The Python program was used to read RFID data, detect motion and send unlock (‘A’) or lock (‘D’) commands to Arduino.
5. Authorized RFID tags were scanned and circular motion was performed for access verification.
6. The experiment was repeated to observe consistent responses between RFID scanning and motion validation.

Circuit Assembly

- Green LED:
  - Anode connected to digital pin 4.
  - Cathode connected to GND
- Red LED:
  - Anode connected to digital pin 3.
  - Cathode connected to GND.
- Servo Motor:

    - Orange wire connected to digital pin 9.
    - Red wire connected to 5V.
    - Brown wire connected to GND.
- RFID Connections:
    - Connected to the laptop via USB cable
- MPU6050 Sensor: Same as Experiment 1

Programming Logic

- The Arduino receives data from both RFID and Python through serial communication.
- When a valid RFID tag is detected, Python prompts the user to perform a motion.
- If the circular motion is correctly detected, Python sends ‘A’ (access granted) to Arduino.
- If the motion is incorrect or RFID is invalid, Python sends ‘D’ (access denied).
- Arduino then activates the servo and LEDs based on the received command.

Code Used

- Arduino

```
#include <MPU6050.h>

#include <Servo.h>

#include <Wire.h>

const int REDLED = 7;

const int GREENLED = 8;


MPU6050 mpu;

Servo myservo;
```

```
const int ACCEL_THRESHOLD = 16400;
const int GYRO_THRESHOLD = 1700;

void setup()
{
  Wire.begin();
  mpu.initialize();
  myservo.attach(9);
  myservo.write(0);
  pinMode(REDLED, OUTPUT);
  pinMode(GREENLED,OUTPUT);
  Serial.begin(9600);
}

void loop()
{
  digitalWrite(REDLED,LOW);
  digitalWrite(GREENLED,LOW);
  int gesture = detectGesture();
  Serial.println(gesture);
  if(Serial.available() > 0)
```

```
  {
    char command = Serial.read();
    if(command == 'a')
    {
      myservo.write(90);
      digitalWrite(GREENLED,HIGH);
      digitalWrite(REDLED,LOW);
      delay(3000);
      myservo.write(0);
    }
    else if(command == 'b')
    {
      myservo.write(0);
      digitalWrite(GREENLED,LOW);
      digitalWrite(REDLED,HIGH);
      delay(3000);
    }
  }
}

int detectGesture()
{
```

```
  int ax,ay,az,gx,gy,gz;

  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  long accel_magnitude_sq = (long)ax*ax + (long)ay*ay + (long)az*az;

  long gyro_magnitude_sq = (long)gx*gx + (long)gy*gy + (long)gz*gz;

  if (accel_magnitude_sq > (long)ACCEL_THRESHOLD * ACCEL_THRESHOLD &&
gyro_magnitude_sq > (long)GYRO_THRESHOLD * GYRO_THRESHOLD)

  {

    return 1; //circle

  }

  else

  {

    return 0;//no circle

  }

}
```

- Python

```
import serial

import time


AUTHORIZED_TAG = "0013084287"

arduino = serial.Serial('COM3', 9600, timeout=1)

print("Serial connection established.")

try:
```

```
while True:
    print("\nPlease scan your card:")
    try:
        uid = input().strip()
    except EOFError:
        continue
    if not uid:
        continue

    print(f"Card Scanned: {uid}")

    if uid == AUTHORIZED_TAG:
        print("RFID authorized.")
        time.sleep(1)

        print("Please do a circular motion. You have 5 seconds...")

        motion_detected = False
        start_time = time.time()
        duration = 5

        try:
```

```
    arduino.flushInput()
    while time.time() - start_time < duration:
        if arduino.in_waiting > 0:
            line = arduino.readline().decode('ascii').strip()
            if line:
                print(f"Received motion data: {line}")
                try:
                    value = int(line)
                    if value == 1:
                        motion_detected = True
                        break
                except ValueError:
                    print(f"Warning: Received non-integer data: '{line}'")
        time.sleep(0.05)
    if motion_detected:
        print("Motion verified. Access granted.")
        arduino.write(b'a')
    else:
        print("Motion not detected in time. Access denied.")
        arduino.write(b'b')
except Exception as e:
    print(f"An error occurred during motion detection: {e}")
```

```
            arduino.write(b'b')
        else:
            print("UID unauthorized. Access denied.")
            arduino.write(b'b')
except KeyboardInterrupt:
    print("\nProgram terminated.")
    arduino.close()
    print("Serial connection closed.")
```

Control Algorithm

1. Initialization
    - Start serial communication at 9600 baud rate.
    - Initialize RFID reader and MPU6050 sensor.
    - Set servo and LEDs as outputs.
2. RFID Scanning
    - Read the RFID tag and verify if UID is authorized.
3. Motion Verification
    - Prompt user to perform circular motion.
    - Check motion data from MPU6050 through Python.
4. Access Control
    - If both RFID and motion are valid, send ‘A’ to unlock and turn on the green LED.
    - If either is invalid, send ‘D’ to deny access and turn on the red LED.

5. Repeat Continuously

    - Continue scanning and verifying for multiple trials.

5.0 DATA COLLECTION

EXPERIMENT 1

This experiment involved an MPU6050 that is a 6-axis motion-tracking device that combines a gyroscope and an accelerometer, used for applications like motion tracking, robotics, and drone flight control. Below is the table showing the wiring details for each component. (See appendix A)

Component	Wiring Details
MPU6050	VCC to 5v, GND to ground, SCL to A5 and SDA to A4. It has a gyroscope and accelerometer attached to the sensor. An accelerometer measures linear acceleration, or changes in velocity, while a gyrometer (or gyroscope) measures angular velocity, or rotational motion.

Data Transmission (Arduino):

- The Arduino code get the acceleration value to define circle gesture

Data Reception (Python):

- The python script uses the pyserial library that can connect a serial connection to the arduino.

- The python reads the data from the arduino via a serial port.

EXPERIMENT 2

This experiment involved an MPU605, servo, USB RFID reader and LED lights. This experiment is to demonstrate a gate system using RFID and gesture tracking Below is the table showing the wiring details for each component. (See appendix B)

Component	**Wiring Details**
MPU6050	VCC to 5v, GND to ground, SCL to A5 and SDA to A4. It has a gyroscope and accelerometer attached to the sensor. An accelerometer measures linear acceleration, or changes in velocity, while a gyrometer (or gyroscope) measures angular velocity, or rotational motion.
Led light	Green LED to D8 and Red Led to D7. Red led will turn ON when the gate is closed and green led will turn ON when gate is open.
Servo motor	The servo motor controls the angle of the arm from 0 to 180 degrees by sending a specific type of electrical signal called Pulse

	Width Modulation (PWM) to the servo's signal pin. The width (duration) of the electrical pulse determines the angle of the servo's shaft. One of the wires is connected to a D9 pin.
USB RFID reader	This component is connected to USB port ‘COM 5’ on the laptop and will only read a specific card manufacturer ‘0013084287, that is located on the card used.

Data Transmission (Arduino):

- The Arduino code gets the acceleration value to define a circle gesture.
- If arduino receives byte ‘a’ from python, the gate will open otherwise closed.

Data Reception (Python):

- The python script uses the pyserial library that can connect a serial connection to the arduino.
- The python reads the data from the arduino via a serial port.
- The python will read RFID card number
- The python will send byte ‘a’ or ‘b’ to arduino

6.0 DATA ANALYSIS

EXPERIMENT 1

From the collected data in the experiment, we analyzed the function of the code from the task given. (See appendix A)

Arduino code:

Action	Description	Output
mpu.getAcceleration(&ax, &ay, &az);	This line of coding was used to get acceleration from MPU	Value acceleration will be read.
long accel_magnitude_sq = (long)ax*ax + (long)ay*ay + (long)az*az; long gyro_magnitude_sq = (long)gx*gx + (long)gy*gy + (long)gz*gz;	As circle is complex in coding, so this equation replace circle gesture based on the complexity of movement to do a circle	Return 1 for circle and return 0 for no circle

Python code:

Action	Description	Output
`plt.xlim(-100,100)` `plt.ylim(-100,100)`	Limit the graph.	Graphs will only be drawn in this range.

`values = line.split(',')`	To split the incoming 4 data.	Read x, y and z axis accelerometer values, and receive a gesture number.

EXPERIMENT 2

From the collected data in the experiment, we analyzed the function of the code from the task given. (See appendix B)

Arduino code:

Action	Description	Output
mpu.getAcceleration(&ax, &ay, &az);	This line of coding was used to get acceleration from MPU	Value acceleration will be read
long accel_magnitude_sq = (long)ax*ax + (long)ay*ay + (long)az*az; long gyro_magnitude_sq = (long)gx*gx + (long)gy*gy + (long)gz*gz;	As circle is complex in coding, so this equation replace circle gesture based on the complexity of movement to do a circle	Return 1 for circle and return 0 for no circle
myservo.attach(9); myservo.write(0);	Attach the servo to pin 9 and make	Servo goes to 0

	it so the flap goes to 0 degrees	degrees.
char command = Serial.read();	This set variable command to read coding from python script.	Either ‘a’ or ‘b’ character

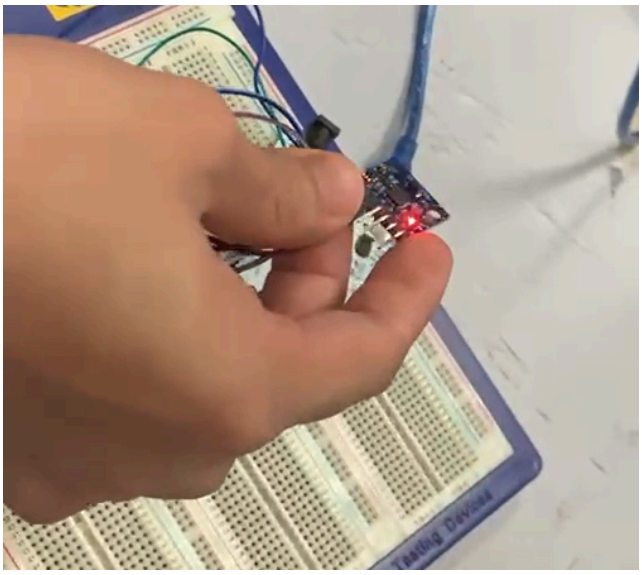
Python code:

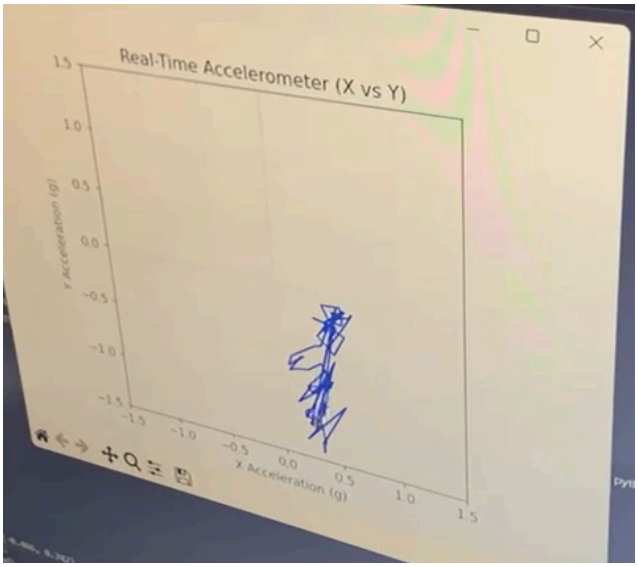
Action	Description	Output
`AUTHORIZED_TAG = "0013084287"`	Set manufacturer card number to the variable	The coding will run if the number matched
`motion_detected = False`	The variable is important whether there is motion or not	Set the variable as false meaning not motion detected yet
`while time.time() - start_time < duration:`	Allow the user to input circular motion within 5 seconds	5 seconds delay to do circle

`if motion_detected:`
`print("Motion verified. Access granted.")`
`arduino.write(b'a')`
	If the variable is true, python script will send ‘a’ character to arduino	Gate will open and green led will lit

7.0 RESULTS

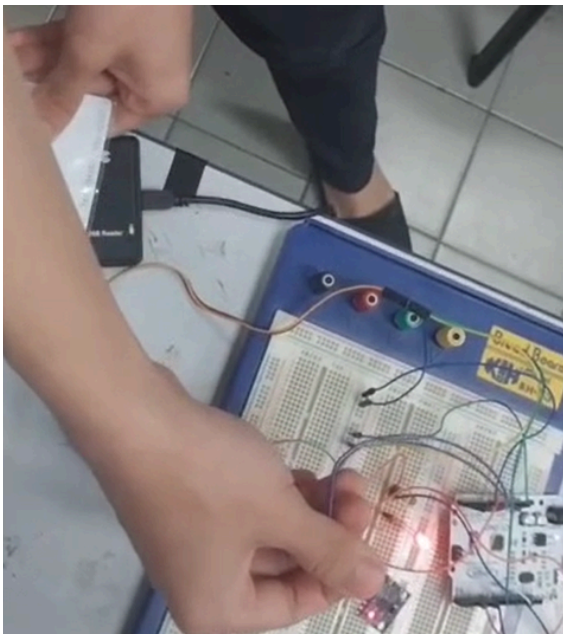
EXPERIMENT 1

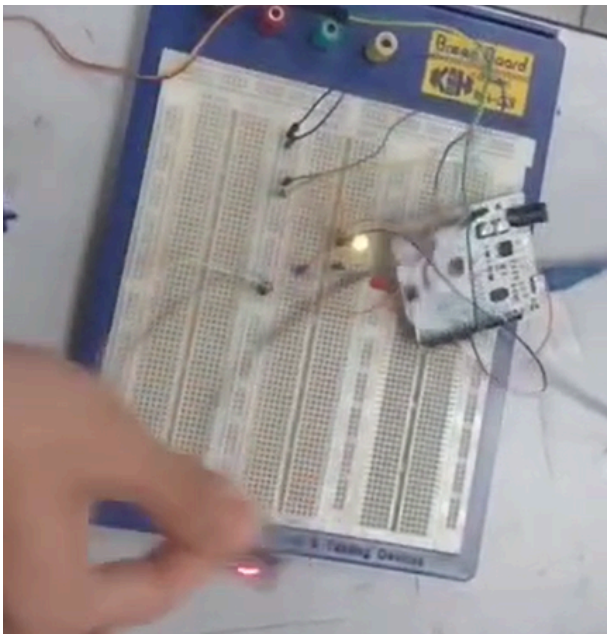




The experiment successfully captured and visualized real-time motion data using the MPU6050 accelerometer sensor. The improved Python script established serial communication with the Arduino and plotted the X-Y accelerometer values dynamically using matplotlib. As the sensor was moved, the plotted path on the graph accurately represented the motion, confirming that real-time data transmission and visualization were functioning correctly. When a circular hand motion was performed, a clear circular pattern appeared on the plot, verifying that the sensor data could effectively detect and represent the gesture. This demonstrated successful integration between Arduino hardware and Python for motion tracking applications.

EXPERIMENT 2





The integrated system combining the RFID module, MPU6050 sensor, servo motor, and LEDs operated as expected. When an RFID tag was scanned, the system verified whether the UID matched an authorized card. If the card was valid, the user was prompted to perform a circular motion, which was analyzed using MPU6050 data. Upon detecting a valid motion pattern, the Arduino received an ‘A’ command from Python, unlocking the servo and turning on the green LED. If the RFID was unauthorized or the motion was incorrect, a ‘D’ command was sent, keeping the servo locked and activating the red LED.

Both hardware and software functioned smoothly, confirming accurate serial communication between Python and Arduino. The system successfully demonstrated the concept of

motion-based authentication, integrating sensing, identification, and actuation in a single automated access control setup.

8.0 DISCUSSION

EXPERIMENT 1

The experiment successfully demonstrated real-time motion detection using the MPU6050 sensor and Arduino. The accelerometer data was sent through serial communication and displayed in Python. The plotted motion path showed that the system could detect circular hand movements, proving the connection between Arduino and Python worked effectively. Although the overall performance was good, the data was sometimes unstable due to small vibrations and inconsistent hand motion during testing.

Sources of Error and Limitations:

- **Sensor Noise:** The MPU6050 produced unstable readings caused by small vibrations and electrical interference.
- **Unsteady Hand Movement:** Irregular hand motion affected the smoothness of the plotted circular path.
- **Serial Delay:** Small delays in serial communication caused data points to skip or appear uneven in Python.
- **Limited Filtering:** The Python code lacked data smoothing, which increased noise in the motion plot.

EXPERIMENT 2

The experiment successfully combined RFID authentication with motion detection to control a servo-based access system. The Arduino and Python communication worked properly, where the system granted access when both the RFID tag was valid and the correct motion pattern was performed. The LEDs responded correctly to indicate access status. However, some delays occurred in motion verification and RFID reading due to communication and timing issues.

Sources of Error and Limitations:

- **RFID Reading Delay:** Detection was slower when the tag was placed too far or at an angle.
- **Signal Interference:** Nearby electronic components caused minor communication interruptions.
- **Serial Communication Delay:** Slight lag between Python and Arduino responses affected system timing.
- **Unfiltered Motion Data:** Noisy sensor data sometimes made motion recognition less accurate.

9.0 CONCLUSION

This experiment successfully demonstrated the integration of multiple sensors and modules, including the MPU6050, RFID, and a servo motor, using serial communication with the Arduino microcontroller. The system was able to detect motion through the MPU6050 sensor, verify user identity using an RFID tag, and control the servo motor and LEDs based on the authentication results. The expected outcome was achieved, as the servo unlocked and the green LED illuminated when a valid RFID tag and correct motion pattern were detected.

Through this experiment, we learned how to combine sensor data processing, serial communication, and actuator control within a single system. It also strengthened our understanding of real-time data exchange between Arduino and Python and highlighted the practical use of motion-based access systems in security and automation applications.

10.0 RECOMMENDATIONS

For future iterations of this experiment, it is recommended to improve the motion detection accuracy by incorporating filtering techniques such as a moving average or Kalman filter to minimize noise from the MPU6050 sensor. This would enhance the reliability of gesture recognition and make the system more responsive. Another improvement would be to implement data logging or visualization features in Python to record sensor values and motion paths for further analysis.

Future students are encouraged to carefully calibrate the MPU6050 sensor before testing to ensure accurate motion readings and to verify RFID tag responses using the serial monitor before integrating the full system. It is also beneficial to use external power for the servo motor to avoid voltage fluctuations. By understanding the communication flow between Arduino and Python early in the process, students can troubleshoot issues more efficiently and gain a deeper appreciation of sensor-actuator integration in mechatronic systems.

11.0 REFERENCES

McWhorter, P. [Paul McWhorter]. (2019, October 31). *9-Axis IMU LESSON 12: Passing Data From Arduino to Python* [Video]. YouTube. https://www.youtube.com/watch?v=8IUHfKKE0tM

shreyas_arbatti. (2021, April 28). Gesture controlled car using NRF24L01 and MPU6050. Arduino Project Hub. https://projecthub.arduino.cc/shreyas_arbatti/gesture-controlled-car-using-nrf24l01-and-mpu6050-8a719a

WaveShapePlay. (2022, July 16). *Arduino and Python Real Time Plot Animation | Lesson 1 Getting Started | PySerial MatPlotLib* [Video]. YouTube. https://www.youtube.com/watch?v=z14l33paZGU

12.0 APPENDICES

Appendix A

```
import serial

import numpy as np

import matplotlib.pyplot as plt

from drawnow import drawnow



arduino = serial.Serial('COM3',115200)

plt.ion()



# Data buffers

x_pos = [0]

y_pos = [0]

z_pos = [0]

vx, vy, vz = 0, 0, 0  # Initial velocities

dt = 0.02  # Time step (~20ms)

last_gesture = "None"

def make_plot():

    plt.title("Real-Time 3D Path")

    plt.xlabel("X Position")
```

```
    plt.ylabel("Y Position")
    plt.plot(x_pos, y_pos, 'b-')
    plt.grid(True)


while True:
    while arduino.in_waiting == 0:
        pass
    line = arduino.readline().decode('utf-8').strip()
    values = line.split(',')
    if len(values) == 4:
        try:
            ax = int(values[0]) / 16384.0  # Convert to g
            ay = int(values[1]) / 16384.0
            az = int(values[2]) / 16384.0 - 1.0  # Remove gravity
            gesture_status = int(values[3])


            if gesture_status == 1:
                last_gesture = "Circle Gesture"
            else:
                last_gesture = "None"
            print(f"Detected Gesture: {last_gesture}")
```

```
        vx += ax * 9.81 * dt
        vy += ay * 9.81 * dt
        vz += az * 9.81 * dt

        x_pos.append(x_pos[-1] + vx * dt)
        y_pos.append(y_pos[-1] + vy * dt)
        z_pos.append(z_pos[-1] + vz * dt)

        # Limit buffer size
        if len(x_pos) > 100:
            x_pos.pop(0)
            y_pos.pop(0)
            z_pos.pop(0)

        drawnow(make_plot)
    except:
        continue
```

Figure E: Python coding for task 1

WEEK4_TASK1.ino

```ino
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

const int ACCEL_THRESHOLD = 16400;
const int GYRO_THRESHOLD = 1700;
int ax,ay,az,gx,gy,gz;

void setup() {
  Serial.begin(115200);
  Wire.begin();
  mpu.initialize();
}

void loop()
{
  mpu.getAcceleration(&ax, &ay, &az);
  int gesture = detectGesture();
  Serial.print(ax);
  Serial.print(",");
  Serial.print(ay);
  Serial.print(",");
  Serial.print(az);
  Serial.print(",");
  Serial.println(gesture);
  delay(600);
}

int detectGesture() {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  long accel_magnitude_sq = (long)ax*ax + (long)ay*ay + (long)az*az;
  long gyro_magnitude_sq = (long)gx*gx + (long)gy*gy + (long)gz*gz;


  if (accel_magnitude_sq > (long)ACCEL_THRESHOLD * ACCEL_THRESHOLD && gyro_magnitude_sq > (long)GYRO_THRESHOLD * GYRO_THRESHOLD)
  {
    return 1;
  }
  else
  {
    return 0;
  }
}
```

Figure F: Arduino coding for task 1

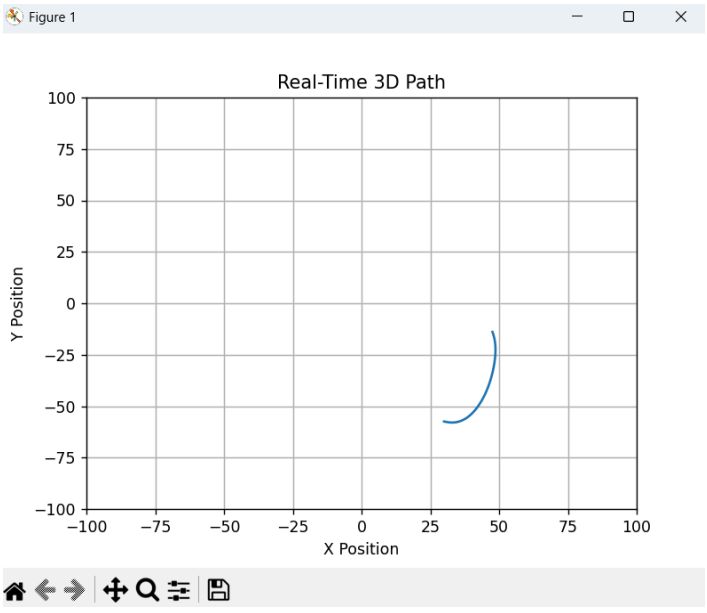


Figure G: Graph plot for circle gesture using python for task 1

Appendix B

```
import serial

import time



AUTHORIZED_TAG = "0013084287"

arduino = serial.Serial('COM3', 9600, timeout=1)

print("Serial connection established.")

try:

    while True:

        print("\nPlease scan your card:")

        try:

            uid = input().strip()

        except EOFError:

            continue

        if not uid:

            continue



        print(f"Card Scanned: {uid}")



        if uid == AUTHORIZED_TAG:

            print("RFID authorized.")

            time.sleep(1)
```

```
            print("Please do a circular motion. You have 5 seconds...")


            motion_detected = False
            start_time = time.time()
            duration = 5


            try:
                arduino.flushInput()
                while time.time() - start_time < duration:
                    if arduino.in_waiting > 0:
                        line = arduino.readline().decode('ascii').strip()
                        if line:
                            print(f"Received motion data: {line}")
                            try:
                                value = int(line)
                                if value == 1:
                                    motion_detected = True
                                    break
                            except ValueError:
                                print(f"Warning: Received non-integer
data: '{line}'")
                    time.sleep(0.05)
```

```
                if motion_detected:
                    print("Motion verified. Access granted.")
                    arduino.write(b'a')
                else:
                    print("Motion not detected in time. Access denied.")
                    arduino.write(b'b')
            except Exception as e:
                print(f"An error occurred during motion detection: {e}")
                arduino.write(b'b')
        else:
            print("UID unauthorized. Access denied.")
            arduino.write(b'b')


except KeyboardInterrupt:
    print("\nProgram terminated.")
    arduino.close()
    print("Serial connection closed.")
```

Figure H: Python coding for task 2

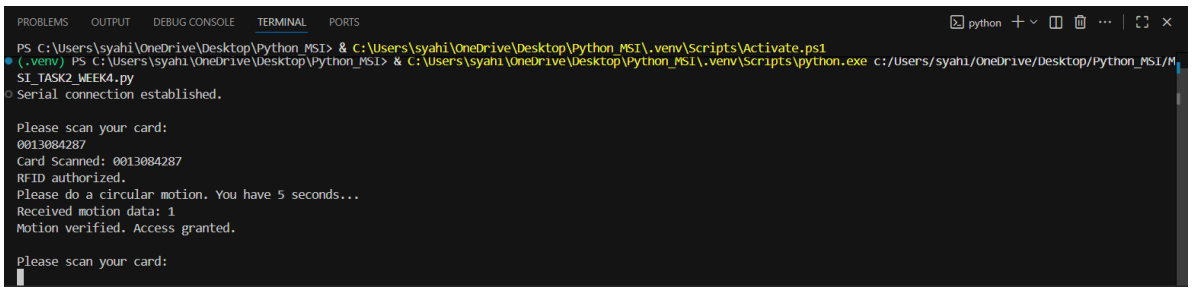


Figure I: Python terminal for task 2

WEEK4_TASK2.ino

```
#include <MPU6050.h>
#include <Servo.h>
#include <Wire.h>
const int REDLED = 7;
const int GREENLED = 8;

MPU6050 mpu;
Servo myservo;

const int ACCEL_THRESHOLD = 16400;
const int GYRO_THRESHOLD = 1700;

void setup()
{
  Wire.begin();
  mpu.initialize();
  myservo.attach(9);
  myservo.write(0);
  pinMode(REDLED, OUTPUT);
  pinMode(GREENLED,OUTPUT);
  Serial.begin(9600);
}

void loop()
{
  digitalWrite(REDLED,LOW);
  digitalWrite(GREENLED,LOW);
  int gesture = detectGesture();
  Serial.println(gesture);
  if(Serial.available() > 0)
  {
    {
      myservo.write(90);
      digitalWrite(GREENLED,HIGH);
      digitalWrite(REDLED,LOW);
      delay(3000);
      myservo.write(0);
    }
    else if(command == 'b')
    {
      myservo.write(0);
      digitalWrite(GREENLED,LOW);
      digitalWrite(REDLED,HIGH);
      delay(3000);
    }
  }
}

int detectGesture()
{
  int ax,ay,az,gx,gy,gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  long accel_magnitude_sq = (long)ax*ax + (long)ay*ay + (long)az*az;
  long gyro_magnitude_sq = (long)gx*gx + (long)gy*gy + (long)gz*gz;
  if (accel_magnitude_sq > (long)ACCEL_THRESHOLD * ACCEL_THRESHOLD && gyro_magnitude_sq > (long)GYRO_THRESHOLD * GYRO_THRESHOLD)
  {
    return 1; //circle
  }
  else
  {
    return 0;//no circle
  }
}
```

Figure J: Arduino code for task 2

13.0 STUDENT'S DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

Signature: 	Read	/
Name: AHMAD HARITH IMRAN BIN MOHD YUSOF	Understand	/
Matric No.: 2311581	Agree	/

Contribution : Assembler, abstract, material and equipment, results, recommendation and student declaration.

Signature: 	Read	/
Name: MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	Understand	/
Matric No.: 2311779	Agree	/
Contribution : Coder, table of content, experimental setup, data collection, data analysis, references and appendices.		

Signature: 	Read	/
Name: ARIFA AQILAH BINTI ABDUL HALIM	Understand	/
Matric No.: 2311530	Agree	/
Contribution : Assembler, introduction, methodology, discussion and conclusion.		